

Yes. Let's make this concrete enough that we can **start coding immediately afterward**.

I suggest we treat this as **MRD v0.2** and freeze the model before choosing packages.

1. Exact Data Model

I would use these entities:

```
User
Exercise
TrainingProgram
ProgramDay
ProgramExercise
WorkoutSession
WorkoutExercise
```

There is deliberately **no separate "Achievement" entity**. The achievement is represented by the actual `WorkoutSession` and its `WorkoutExercise` records.

1.1 User

```
dart

class User {
  String id;
  String name;
  String? email;
  String? photoUrl;

  DateTime? birthDate;
  String? gender;

  FitnessLevel fitnessLevel;
  List<TrainingGoal> goals;

  String language; // "he", "en"
  bool active;

  DateTime createdAt;
  DateTime updatedAt;
}
```

Enums

```
FitnessLevel
└─ beginner
```

- └ intermediate
- └ advanced

TrainingGoal

- └ generalFitness
- └ strength
- └ mobility
- └ cardio
- └ weightManagement

2. Exercise

The exercise is the **master definition**.

It should NOT contain information such as "12 repetitions" because that belongs to a program.

```
dart

class Exercise {
  String id;

  LocalizedText name;
  LocalizedText description;
  LocalizedText instructions;

  String? photoUrl;

  ExerciseCategory category;
  Difficulty difficulty;

  List<String> targetMuscles;

  Equipment equipment;

  int? defaultDurationSeconds;
  int? defaultRepetitions;

  bool active;

  DateTime createdAt;
  DateTime updatedAt;
}
```

LocalizedText

Rather than having:

```
nameHebrew  
nameEnglish
```

I recommend:

```
dart  
  
class LocalizedText {  
  String he;  
  String en;  
}
```

Later we can add more languages without changing the model.

3. Exercise Category

```
ExerciseCategory  
├─ warmup  
├─ mobility  
├─ strength  
├─ cardio  
├─ stretching  
└─ cooldown
```

4. Difficulty

```
Difficulty  
├─ beginner  
├─ intermediate  
└─ advanced
```

5. Equipment

Initially:

```
Equipment  
└─ none
```

But the model supports future expansion:

```
Equipment
├─ none
├─ chair
├─ mat
├─ resistanceBand
├─ dumbbells
└─ ...
```

6. Training Program

A program belongs to a user.

```
dart

class TrainingProgram {
  String id;

  String userId;

  String name;
  String? description;

  DateTime startDate;
  DateTime? endDate;

  ProgramScheduleType scheduleType;

  bool active;

  DateTime createdAt;
  DateTime updatedAt;
}
```

Schedule type

```
ProgramScheduleType
├─ daily
├─ weekly
└─ custom
```

7. Program Day

A program contains days.

dart

```
class ProgramDay {  
  String id;  
  
  String programId;  
  
  String title;  
  String? description;  
  
  DateTime? date;  
  int? weekday;  
  
  int order;  
  
  List<ProgramExercise> exercises;  
}
```

Why both `date` and `weekday` ?

Because we want to support:

Fixed date

28/08/2026

and repeating weekly:

Monday
Wednesday
Friday

For weekly programs:

```
weekday = 1 // Monday  
weekday = 3 // Wednesday  
weekday = 5 // Friday
```

`date` remains null.

8. Program Exercise

This is one of the most important entities.

dart

```
class ProgramExercise {
  String id;

  String programDayId;
  String exerciseId;

  int order;

  int? sets;
  int? repetitions;

  int? durationSeconds;
  int? restSeconds;

  String? notes;
}
```

Example:

```
Squat

sets: 2
repetitions: 12
duration: null
rest: 30
```

Another:

```
Plank

sets: 2
repetitions: null
duration: 30
rest: 30
```

This gives us both **repetition-based** and **time-based** exercises.

9. Workout Session

This is an actual workout performed by the user.

```
dart

class WorkoutSession {
  String id;

  String userId;
  String programDayId;
```

```

DateTime startedAt;
DateTime? completedAt;

WorkoutStatus status;

int? durationSeconds;

String? notes;

List<WorkoutExercise> exercises;
}

```

Status

```

WorkoutStatus
├─ started
├─ completed
├─ partiallyCompleted
├─ skipped
└─ cancelled

```

10. Workout Exercise

This records the **actual achievement**.

```

dart

class WorkoutExercise {
  String id;

  String workoutSessionId;
  String exerciseId;

  int? plannedSets;
  int? actualSets;

  int? plannedRepetitions;
  int? actualRepetitions;

  int? plannedDurationSeconds;
  int? actualDurationSeconds;

  bool completed;

  String? notes;
}

```

However, I recommend one improvement here.

For proper tracking, we should record individual sets.

11. Workout Set

Therefore I would add:

```
dart

class WorkoutSet {
  String id;

  String workoutExerciseId;

  int setNumber;

  int? plannedRepetitions;
  int? actualRepetitions;

  int? plannedDurationSeconds;
  int? actualDurationSeconds;

  bool completed;
}
```

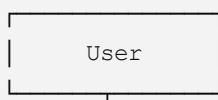
Then:

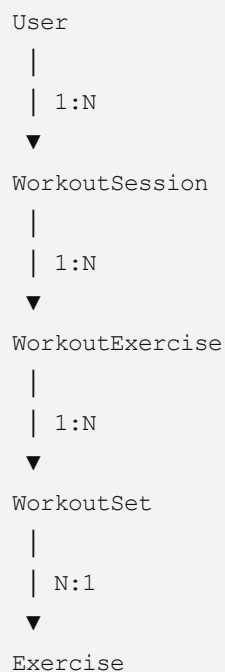
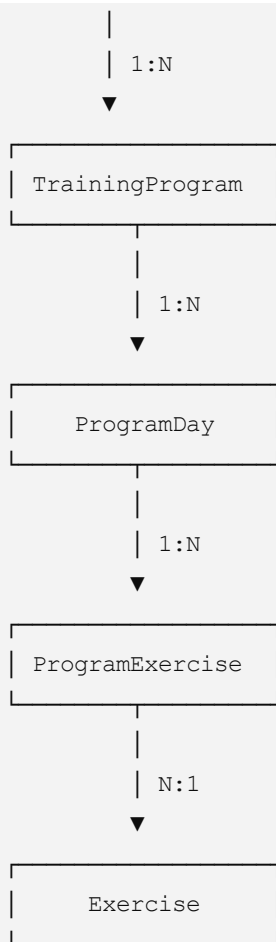
```
WorkoutSession
├──
└── WorkoutExercise
    ├──
    ├── WorkoutSet 1
    ├── WorkoutSet 2
    └── WorkoutSet 3
```

This is considerably better for progress tracking.

12. Final Data Model

So the complete relationship becomes:





This is the model I recommend freezing.

13. Screens

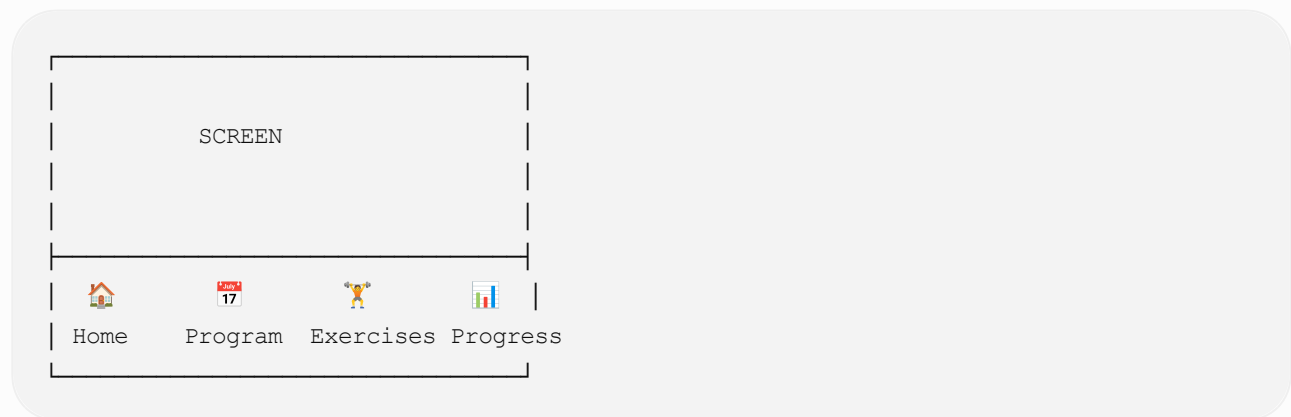
Now the application itself.

I would divide it into **User Mode** and **Admin/Trainer Mode**, but initially we can implement only User Mode.

13.1 Application Navigation

Mobile

Bottom navigation:



Four main sections:

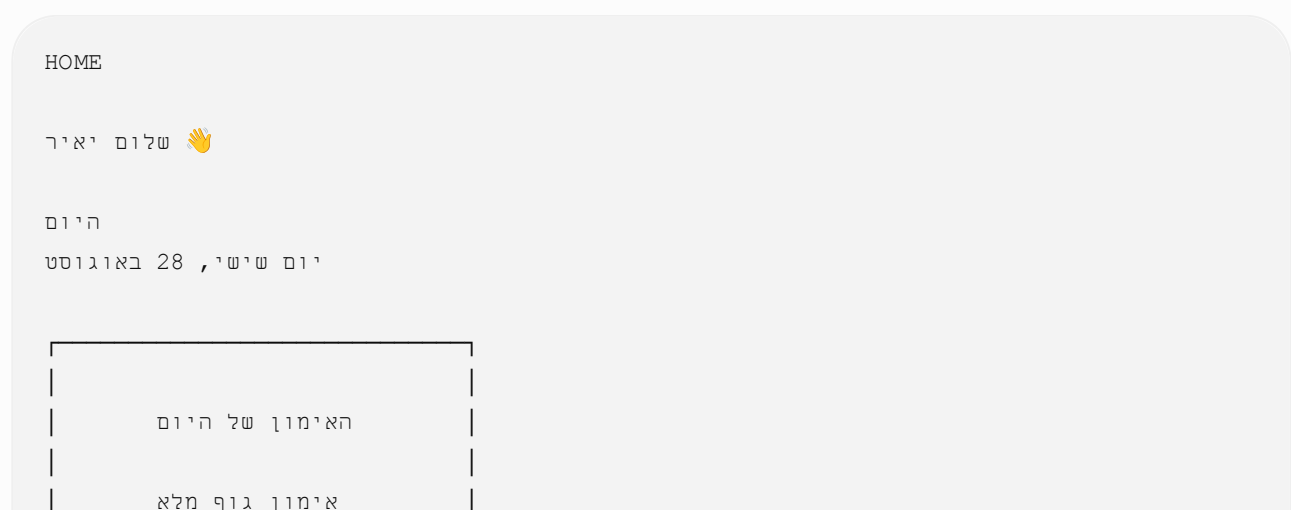
1. Home
2. Program
3. Exercises
4. Progress

Profile can be accessed from the Home/AppBar.

14. Home Screen

Purpose:

Tell the user what to do now.



דקות 20
תרגילים 8

[התחל אימון]

השבוע

- ✓ שני
- ✓ רביעי
- שישי
- שבת

התקדמות
אימונים 3
דקות 61

Possible states

No workout today

אין אימון מתוכנן להיום
[הצג את התוכנית שלי]

Workout completed

✓ האימון של היום הושלם
דקות 20
100%
[הצג סיכום]

15. Program Screen

Shows the user's current training program.

התוכנית שלי

20-Minute Home Workout

28/08/2026 – 28/09/2026

השבוע

✓ שני

Mobility

✓ רביעי

Strength

● שישי

Full Body

○ ראשון

Cardio

Tap a day:

שישי – Full Body

20 דקות

1. חימום
2. Squat
3. Wall Push-up
4. Reverse Lunge
5. Glute Bridge
6. Plank
7. Cardio
8. Cool-down

[התחל אימון]

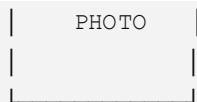
16. Exercise Library

תרגילים

[ חיפוש]

קטגוריות

[אירובי] [כוח] [הכול]
[מתיחות] [ניידות]



סקווט

כוח • מתחילים



גשר ישבן

כוח • מתחילים

17. Exercise Details



סקווט

מתחילים

שרירים:

ירכיים • ישבן

ציוד:

ללא ציוד

איך לבצע

- עמוד עם רגליים ברוחב הכתפיים.
- דחוף את האגן לאחור.
- כופף את הברכיים.
- חזור לעמידה.

דגשים

...שמור על גב ניטרלי

18. Start Workout

This deserves special attention.

The user shouldn't have to constantly read instructions.

Before starting

אימון גוף מלא

20 דקות

8 תרגילים

היום:

28/08/2026

[התחל]

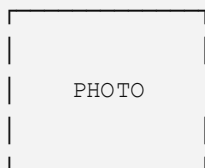
19. Active Workout

For example:

אימון גוף מלא

תרגיל 2 מתוך 8

סקוואט



חזרות 12

סטים 2

[התחל סט]

20. Active Set

For a repetition exercise:

סקווט

סט 1 מתוך 2

12

חזרות

[סיימתי ✓]

After pressing:

סט 1 ✓

איך היה?

- ☐ קל
- ☒ טוב
- ☐ קשה

[המשך]

21. Timed Exercise

For example plank:

פלאנק

סט 1 מתוך 2

00:30



[התחל]

After completion:

הושלם ✓

שניות 30

[המשך]

22. Rest Screen

Instead of simply waiting:

מנוחה

00:23

התרגיל הבא:

שכיבות סמיכה

[דלג]

This makes the workout feel like a real guided workout.

23. Workout Completion

🎉 כל הכבוד!

האימון הושלם

20:14

8 / 8 תרגילים

100%

12 / 12 סקוויט

10 / 10 שכיבות סמיכה

10 / 10 מכרעים

...

[סיום]

24. Partial Completion

If the user stops:

האימון הופסק

השלמת:

תרגילים 8 / 5

דקות 14

[שמור וסיים]

[המשך אימון]

The session should be stored as:

```
status = partiallyCompleted
```

25. Progress Screen

ההתקדמות שלי

השבוע

3	61	92%
השלמה	דקות	אימונים

אימונים אחרונים

28/08 ✓ Full Body

26/08 ✓ Mobility

24/08 ✓ Strength

21/08 80% Cardio

Later we can add graphs.

26. Profile

הפרופיל שלי

[PHOTO]

י איר

רמת כושר

מתחזיל

מטרות

- ✓ כושר כללי
- ✓ נייחות

שפה

עברית

הגדרות

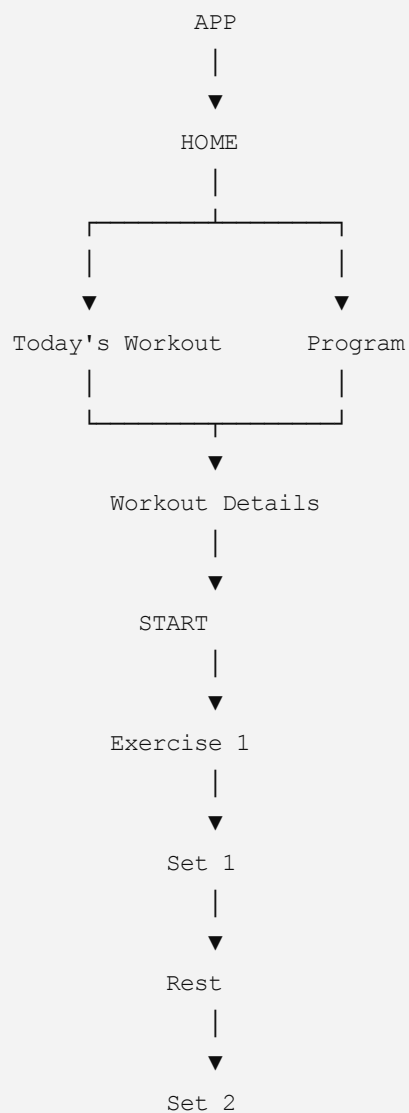
התראות

יחידות

שפה

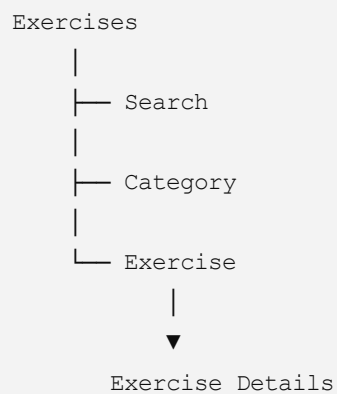
27. Screen Flow

The normal user journey:

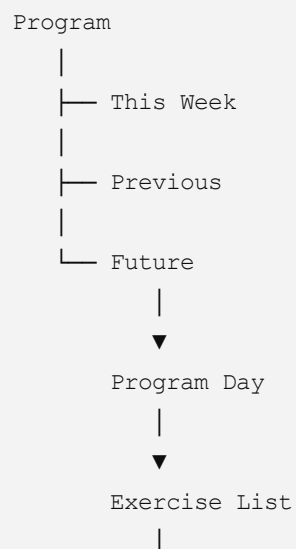




28. Exercise Discovery Flow



29. Program Flow



30. What happens when a workout is started?

This is important from a technical perspective.

Step 1

Create:

```
WorkoutSession  
status = started  
startedAt = NOW
```

Step 2

Copy the relevant program exercises into the session.

We do not modify the program.

Step 3

For every exercise:

```
WorkoutExercise
```

is created.

Step 4

For every planned set:

```
WorkoutSet
```

is created.

Step 5

The user performs the workout.

Actual values are recorded.

Step 6

When finished:

```
WorkoutSession.status = completed
completedAt = NOW
```

This gives us a permanent snapshot of what happened.

31. Important Design Decision: Snapshot

Suppose today the program says:

```
Squat
2 × 12
```

Next month the trainer changes it to:

```
Squat
3 × 15
```

The old workout must **still show 2 × 12**.

Therefore the Workout Session stores the planned values at the time the workout started.

This is why we need:

```
Program
  ↓
ProgramExercise

then at workout start:

ProgramExercise
  ↓
WorkoutExercise
  ↓
WorkoutSet
```

This is a very important part of the architecture.

32. MVP Screen List

I would freeze the first version at these screens:

1. Home
2. My Program
3. Program Day

- 4. Exercise Library
- 5. Exercise Details
- 6. Workout Introduction
- 7. Active Exercise
- 8. Rest
- 9. Workout Summary
- 10. Workout History
- 11. Workout Details
- 12. Progress
- 13. Profile
- 14. Settings

Admin screens — Phase 2

- 15. Admin Dashboard
- 16. Exercise Management
- 17. Exercise Editor
- 18. User Management
- 19. User Details
- 20. Program Management
- 21. Program Editor
- 22. Program Assignment

33. Recommended MVP Architecture

I would now use:

```
Flutter
|
├─ presentation
|   ├── home
|   ├── program
|   ├── exercises
|   ├── workout
|   ├── history
|   ├── progress
|   └─ profile
|
├─ domain
|   ├── entities
|   └─ repositories
|
├─ data
|   ├── models
|   ├── database
|   └─ repositories
|
```

```
└─ core
  ├── localization
  ├── theme
  ├── routing
  └─ utilities
```

For state management, I'd lean toward **Riverpod**.

For the local database, I'd lean toward **Drift + SQLite**, because the relationships above are naturally relational and we will eventually want queries such as:

```
Give me:
  all workouts
  for user X
  during August
  grouped by week
```

SQLite handles that extremely well.

One more thing before coding

I think we should add **two small requirements** to the MRD now:

A. Exercise media

Don't limit ourselves to `photoUrl`.

```
ExerciseMedia
├─ id
├─ exerciseId
├─ type          // photo / video
├─ url
├─ thumbnail
└─ order
```

This lets an exercise eventually have several photos or a demonstration video.

B. User feedback per exercise

During the workout:

```
Difficulty:
1  2  3  4  5

Pain:
None / Mild / Significant
```

I would **not implement pain tracking in the first MVP**, but I'd leave room for it because it could become important for adapting programs.

My recommendation

At this point I would freeze the **core model** as:

User → TrainingProgram → ProgramDay → ProgramExercise → Exercise

and

User → WorkoutSession → WorkoutExercise → WorkoutSet → Exercise

Then the next step should be to turn this into the actual **Flutter project specification**:

- exact SQLite/Drift tables
- Dart model classes
- repository interfaces
- Riverpod providers
- routing
- complete folder structure
- RTL/i18n setup
- responsive mobile/desktop UI rules

That will put us directly at the point where we can create the project and start writing code.